# PROJECT REPORT

## EXPERIMENT - 10

### PROJECT TITLE

## Industrial Diesel Generator Monitoring and Maintenance System


**Submitted by**

MUTHU KRISHNA P - 24MER033

PRIYANTH C S - 24MER044

SRI RAMAN G - 24MEL068

PRADEEP P - 24MER041

## 10.Industrial Diesel Generator Monitoring and Maintenance System

### 1. Project Overview

The **Industrial Diesel Generator Monitoring and Maintenance System** is an IoT-based system developed to continuously monitor the operating condition of a diesel generator. Diesel generators are commonly used as backup power sources in industries, hospitals, factories, data centers, commercial buildings, and telecommunication facilities.

A generator may experience problems such as **engine overheating, excessive vibration, low oil pressure, low fuel level, abnormal battery voltage, and unexpected shutdown**. If these conditions are not identified at an early stage, they can cause serious engine damage, increased maintenance costs, and interruption of power supply.

The proposed system uses an **ESP8266 NodeMCU** as the main controller. Sensors are connected to the controller to measure important generator parameters. The ESP8266 processes the sensor readings, compares them with predefined safety limits, displays the information locally, activates an alarm when an abnormal condition occurs, and can transmit the information to a remote monitoring system through Wi-Fi.

### 2. Problem Statement

Diesel generators are critical machines in industrial environments. Failure of a generator can interrupt production and essential services. Traditional generator monitoring mainly depends on manual inspection or periodically checking the generator control panel.

Manual inspection cannot continuously monitor every parameter. A technician may not immediately notice increasing engine temperature, excessive vibration, low oil pressure, low fuel level, or declining battery voltage.

Another problem is that conventional preventive maintenance is often performed according to fixed schedules. This does not always reflect the actual operating condition of the generator. A heavily used generator may require maintenance earlier than expected.

Therefore, an intelligent monitoring system is required to continuously collect generator operating parameters, identify abnormal conditions, provide alarms, and make information available to maintenance personnel.

### 3. Proposed Solution

The proposed solution is an **ESP8266-based Industrial Diesel Generator Monitoring and Maintenance System**.

Multiple sensors are connected to the ESP8266 to measure important operating parameters. The controller continuously collects the sensor data and compares each parameter with predefined threshold values.

The main monitoring parameters are:

- Engine temperature
- Vibration
- Oil pressure
- Fuel level
- Battery voltage
- Generator running status

When the system detects an abnormal condition, it generates an alert through a buzzer and displays the fault condition on an OLED display. Through Wi-Fi, the measured data can also be sent to a cloud or MQTT monitoring platform.

**4. System Architecture**

The system architecture consists of sensing, processing, communication, monitoring, and maintenance layers.

**4.1 Generator Layer**

The diesel generator is the equipment being monitored.

Important parameters include:

- Engine temperature
- Engine vibration
- Lubricating-oil pressure
- Fuel level
- Battery voltage
- Generator operating status

**4.2 Sensor Layer**

Sensors collect information from the generator.

Typical prototype sensors include:

- Temperature sensor
- Vibration sensor
- Oil-pressure sensor
- Fuel-level sensor
- Voltage-sensing circuit
- Generator-status sensor

**4.3 Processing Layer**

The **ESP8266 NodeMCU** acts as the central controller.

Its responsibilities are:

- Reading sensor values
- Processing sensor signals
- Comparing values with thresholds
- Detecting abnormal conditions
- Controlling the buzzer
- Updating the display
- Sending data through Wi-Fi

**4.4 Communication Layer**

The ESP8266 uses its built-in Wi-Fi capability to transmit generator data to a remote server or IoT platform.

**4.5 Monitoring Layer**

The OLED provides local information, while a cloud or MQTT dashboard can provide remote monitoring.

**4.6 Maintenance Layer**

The maintenance team can use the information to:

- Identify abnormal conditions.
- Plan inspections.
- Schedule maintenance.
- Analyze historical trends.
- Reduce unexpected downtime.

**5. Circuit Table**

The following circuit table is suitable for an **ESP8266 NodeMCU prototype**.

| **Component** | **Pin/Output** | **ESP8266 Connection** | **Purpose** |
|---|---|---|---|
| DS18B20 Temperature Sensor | DATA | D3 / GPIO0 | Engine temperature |
| DS18B20 | VCC | 3.3V | Power |
| DS18B20 | GND | GND | Ground |

| Component | Pin/Output | ESP8266 Connection | Purpose |
|---|---|---|---|
| SW-420 Vibration Sensor | DO | D5 / GPIO14 | Vibration detection |
| SW-420 | VCC | 3.3V | Power |
| SW-420 | GND | GND | Ground |
| Oil Pressure Sensor | Analog | External ADC | Oil pressure |
| Fuel Level Sensor | Analog | External ADC | Fuel level |
| Battery Voltage Sensor | Analog | External ADC | Battery voltage |
| Generator Status | Digital | D6 / GPIO12 | Generator ON/OFF |
| OLED | SDA | D2 / GPIO4 | $I^2C$ data |
| OLED | SCL | D1 / GPIO5 | $I^2C$ clock |
| OLED | VCC | 3.3V | Power |
| OLED | GND | GND | Ground |
| Buzzer | Signal | D7 / GPIO13 | Alarm |
| Buzzer | GND | GND | Ground |

**Important ESP8266 consideration**

The ESP8266 has only **one built-in analog input (A0)**. Since oil pressure, fuel level, and battery voltage are analog measurements, an external ADC such as an **ADS1115** is recommended for a complete prototype.

All voltage signals must be appropriately conditioned before entering the ESP8266. A generator's 12 V/24 V battery or other high-voltage/high-energy circuits must **never be connected directly** to an ESP8266 input.

**6. Circuit Design**

The circuit is designed around the ESP8266 NodeMCU. Sensors collect generator parameters and send their outputs to the controller.

The proposed circuit uses the ESP8266 NodeMCU as the central processing unit of the Industrial Diesel Generator Monitoring and Maintenance System. The DS18B20 temperature sensor is connected to a digital GPIO pin to measure the generator's temperature. An SW-420 vibration sensor is connected to another digital input to detect abnormal mechanical vibration. A generator-status sensor is used to identify whether the generator is operating.

The oil-pressure sensor, fuel-level sensor, and battery-voltage monitoring circuit are connected to an external ADS1115 ADC because the ESP8266 has limited analog-input capability. The ADS1115 converts the analog sensor signals into digital values that can be processed by the ESP8266.

An OLED display is connected through the $I^2C$ interface and displays temperature, vibration, oil pressure, fuel level, battery voltage, and system status. A buzzer provides an immediate local warning when a critical operating condition is detected. The ESP8266's built-in Wi-Fi capability can transmit the collected data to a cloud or MQTT server for remote monitoring.

**7. Algorithm**

**Step-by-step algorithm**

**Step 1:** Start the system.

**Step 2:** Initialize the ESP8266.

**Step 3:** Initialize all sensors.

**Step 4:** Initialize the OLED display and buzzer.

**Step 5:** Connect to Wi-Fi.

**Step 6:** Read engine temperature.

**Step 7:** Read vibration status.

**Step 8:** Read oil pressure.

**Step 9:** Read fuel level.

**Step 10:** Read battery voltage.

**Step 11:** Read generator operating status.

**Step 12:** Process all sensor measurements.

**Step 13:** Compare each measurement with its predefined threshold.

**Step 14:** Determine system status.

**Step 15:** Display the measurements on the OLED.

**Step 16:** Activate the buzzer when an abnormal condition is detected.

**Step 17:** Send data to the cloud/MQTT server.

**Step 18:** Store or display historical information.

**Step 19:** Repeat the monitoring cycle.

**8. Coding**

The following Arduino IDE code is suitable as a **prototype program for ESP8266**.

```
#include <ESP8266WiFi.h>
#include <DHT.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// ====================================================
// WIFI SETTINGS
// ====================================================

#define WIFI_SSID     "Pradeep"
#define WIFI_PASSWORD "12345678"

// ====================================================
// ADAFRUIT IO SETTINGS
// ====================================================

#define AIO_SERVER      "io.adafruit.com"
#define AIO_SERVERPORT  1883

#define AIO_USERNAME    "muthumkt"
#define AIO_KEY         "aio_Dpts45gXpVu4he3HHcwUXVBgOuKb"

// ====================================================
// SENSOR MODE
// 1 = ACS712 CURRENT
// 2 = VOLTAGE SENSOR
// ====================================================
```

```
#define SENSOR_MODE 1

// ====================================================
// ANALOG INPUT
// ====================================================

#define ANALOG_PIN A0

// ====================================================
// DHT11
// ====================================================

#define DHT_PIN  D7
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

// ====================================================
// BUZZER
// ====================================================

#define BUZZER_PIN D6

// ====================================================
// ALARM LIMITS
// ====================================================

float MAX_CURRENT = 10.0;
float MAX_VOLTAGE = 250.0;
float MAX_TEMPERATURE = 60.0;
```

```
// ====================================================
// ACS712 SETTINGS
// ====================================================

// ACS712 5A  = 0.185
// ACS712 20A = 0.100
// ACS712 30A = 0.066

float ACS_SENSITIVITY = 0.100;

// IMPORTANT:
// This value must be calibrated for your sensor.
// For this simple example it is set to 0.50 V
// because ESP8266 A0 is being treated as 0-1V.
//
// If your ACS712 output is scaled differently,
// calibration is required.

float ACS_ZERO_VOLTAGE = 0.50;

// ====================================================
// VOLTAGE SENSOR SETTINGS
// ====================================================

float VOLTAGE_RATIO = 5.0;

// ====================================================
// WIFI CLIENT
// ====================================================
```

```
WiFiClient client;

// ====================================================
// MQTT CLIENT
// ====================================================

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// ====================================================
// ADAFRUIT IO FEEDS
// ====================================================

Adafruit_MQTT_Publish currentFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/current"
  );

Adafruit_MQTT_Publish voltageFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/voltage"
  );
```

```
Adafruit_MQTT_Publish temperatureFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/temperature"
  );

Adafruit_MQTT_Publish humidityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/humidity"
  );

// ================================================
// CONNECT WIFI
// ================================================

void connectWiFi()
{
  if (WiFi.status() == WL_CONNECTED)
  {
    return;
  }

  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
 while (WiFi.status() != WL_CONNECTED)
 {
  delay(500);
  Serial.print(".");
 }

 Serial.println();
 Serial.println("WiFi Connected!");

 Serial.print("IP Address: ");
 Serial.println(WiFi.localIP());
}

// ====================================================
// CONNECT ADAFRUIT IO
// ====================================================

void connectMQTT()
{
 if (mqtt.connected())
 {
  return;
 }

 Serial.println("Connecting to Adafruit IO...");

 int8_t ret;

 while ((ret = mqtt.connect()) != 0)
 {
```

```
   Serial.print("MQTT Error: ");
   Serial.println(mqtt.connectErrorString(ret));

   mqtt.disconnect();

   delay(5000);
  }

 Serial.println("Adafruit IO Connected!");
}

// ====================================================
// READ CURRENT FROM ACS712
// ====================================================

float readCurrent()
{
 long total = 0;

 for (int i = 0; i < 100; i++)
 {
  total += analogRead(ANALOG_PIN);
  delay(2);
 }

 float averageADC = total / 100.0;

 // ESP8266 A0 assumed 0-1V
 float sensorVoltage =
  (averageADC / 1023.0) * 1.0;
```

```
  float current =
    (sensorVoltage - ACS_ZERO_VOLTAGE) /
    ACS_SENSITIVITY;

  if (current < 0)
  {
    current = -current;
  }

  // Remove very small noise
  if (current < 0.05)
  {
    current = 0;
  }

  return current;
}

// ====================================================
// READ VOLTAGE SENSOR
// ====================================================

float readVoltage()
{
  long total = 0;

  for (int i = 0; i < 50; i++)
  {
    total += analogRead(ANALOG_PIN);
```

```
  delay(2);
 }

 float averageADC = total / 50.0;

 float sensorVoltage =
  (averageADC / 1023.0) * 1.0;

 float actualVoltage =
  sensorVoltage * VOLTAGE_RATIO;

 if (actualVoltage < 0)
 {
  actualVoltage = 0;
 }

 return actualVoltage;
}

// ================================================
// SETUP
// ================================================

void setup()
{
 Serial.begin(115200);

 // Start DHT11
 dht.begin();
```

```
  // Buzzer
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  delay(2000);

  // WiFi
  connectWiFi();

  // Adafruit IO
  connectMQTT();

  Serial.println();
  Serial.println("========================================");
  Serial.println(" INDUSTRIAL DIESEL GENERATOR");
  Serial.println(" MONITORING SYSTEM");
  Serial.println("========================================");
  Serial.println("DHT11      : READY");
  Serial.println("Buzzer     : READY");
  Serial.println("WiFi       : CONNECTED");
  Serial.println("Adafruit IO : CONNECTED");

#if SENSOR_MODE == 1

  Serial.println("Analog Mode : ACS712 CURRENT");

#else

  Serial.println("Analog Mode : VOLTAGE SENSOR");
```

```
#endif

 Serial.println("=========================================");
 Serial.println();
}

// ====================================================
// MAIN LOOP
// ====================================================

void loop()
{
 // ====================================================
 // WIFI
 // ====================================================

 if (WiFi.status() != WL_CONNECTED)
 {
  connectWiFi();
 }

 // ====================================================
 // ADAFRUIT IO
 // ====================================================

 connectMQTT();

 // ====================================================
 // READ DHT11
 // ====================================================
```

```
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

// ==================================================
// DHT11 ERROR
// ==================================================

if (isnan(temperature) || isnan(humidity))
{
  Serial.println();
  Serial.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
  Serial.println("DHT11 SENSOR ERROR");
  Serial.println("Check sensor connections.");
  Serial.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");

  digitalWrite(BUZZER_PIN, LOW);

  delay(2000);

  return;
}

// ==================================================
// DISPLAY SENSOR DATA IN SERIAL MONITOR
// ==================================================

Serial.println();
Serial.println("----------------------------------------");
```

```
  Serial.print("Temperature : ");
  Serial.print(temperature, 1);
  Serial.println(" C");

  Serial.print("Humidity    : ");
  Serial.print(humidity, 1);
  Serial.println(" %");

  // ================================================
  // ANALOG SENSOR
  // ================================================

  float current = 0.0;
  float voltage = 0.0;

#if SENSOR_MODE == 1

  current = readCurrent();

  Serial.print("Current     : ");
  Serial.print(current, 2);
  Serial.println(" A");

#else

  voltage = readVoltage();

  Serial.print("Voltage     : ");
  Serial.print(voltage, 2);
  Serial.println(" V");
```

```
#endif

 // ================================================
 // ALARM
 // ================================================

 bool alarm = false;

 if (temperature >= MAX_TEMPERATURE)
 {
   alarm = true;
 }

#if SENSOR_MODE == 1

 if (current >= MAX_CURRENT)
 {
   alarm = true;
 }

#else

 if (voltage >= MAX_VOLTAGE)
 {
   alarm = true;
 }

#endif
```

```
// ================================================
// BUZZER
// ================================================

if (alarm)
{
  digitalWrite(BUZZER_PIN, HIGH);
  Serial.println("STATUS     : WARNING");
  Serial.println("BUZZER     : ON");
}
else
{
  digitalWrite(BUZZER_PIN, LOW);
  Serial.println("STATUS     : NORMAL");
  Serial.println("BUZZER     : OFF");
}

// ================================================
// SEND TEMPERATURE
// ================================================

if (temperatureFeed.publish(temperature))
{
  Serial.println("Temperature : SENT TO ADAFRUIT IO");
}
else
{
  Serial.println("Temperature : SEND FAILED");
}
```

```
 // ================================================
 // SEND HUMIDITY
 // ================================================

 if (humidityFeed.publish(humidity))
 {
  Serial.println("Humidity    : SENT TO ADAFRUIT IO");
 }
 else
 {
  Serial.println("Humidity    : SEND FAILED");
 }

 // ================================================
 // SEND CURRENT
 // ================================================

#if SENSOR_MODE == 1

 if (currentFeed.publish(current))
 {
  Serial.println("Current     : SENT TO ADAFRUIT IO");
 }
 else
 {
  Serial.println("Current     : SEND FAILED");
 }

#endif
```

```
  // ================================================
  // SEND VOLTAGE
  // ================================================

#if SENSOR_MODE == 2

  if (voltageFeed.publish(voltage))
  {
    Serial.println("Voltage     : SENT TO ADAFRUIT IO");
  }
  else
  {
    Serial.println("Voltage     : SEND FAILED");
  }

#endif

  Serial.println("-----------------------------------------");
  Serial.println();

  // DHT11 / Adafruit IO update interval
  delay(5000);
}
```

**9. System Working**

When the system is powered on, the ESP8266 initializes all sensors, the OLED display, and the alarm circuit. It then connects to the available Wi-Fi network.

After establishing communication, the controller continuously reads the generator parameters.

**Temperature monitoring**

The temperature sensor measures engine temperature. If the temperature rises above the configured limit, the system changes the status to **WARNING** or **CRITICAL**.

**Vibration monitoring**

The vibration sensor detects abnormal mechanical vibration. Continuous or excessive vibration can indicate problems such as imbalance, loose components, or mechanical wear.

**Oil-pressure monitoring**

The oil-pressure sensor measures the lubrication pressure. If pressure falls below the minimum acceptable level, the system generates a critical alarm.

**Fuel monitoring**

The fuel-level sensor determines the remaining fuel. If the fuel level becomes too low, the system generates a warning to indicate that refueling is required.

**Battery monitoring**

The battery-voltage monitoring circuit checks the generator battery. A low voltage condition can indicate a weak battery or charging-system problem.

## 10. Bill of Materials

| S.No. | Component | Quantity | Approx. Cost |
|---|---|---|---|
| 1 | ESP8266 NodeMCU | 1 | ₹300–₹500 |
| 2 | DS18B20 Temperature Sensor | 1 | ₹100–₹200 |
| 3 | SW-420 Vibration Sensor | 1 | ₹50–₹100 |
| 4 | Oil Pressure Sensor | 1 | ₹300–₹1,000 |
| 5 | Fuel Level Sensor | 1 | ₹150–₹500 |
| 6 | ADS1115 External ADC | 1 | ₹150–₹300 |
| 7 | OLED 0.96-inch $I^2C$ | 1 | ₹150–₹300 |
| 8 | Buzzer | 1 | ₹10–₹30 |
| 9 | Breadboard | 1 | ₹80–₹150 |
| 10 | Jumper Wires | 1 set | ₹50–₹150 |
| 11 | Resistors | 1 set | ₹20–₹50 |
| 12 | Power Supply | 1 | ₹100–₹300 |

| S.No. | Component | Quantity | Approx. Cost |
|---|---|---|---|
| 13 | USB Cable | 1 | ₹50–₹150 |

**Estimated prototype cost**

**Approximately ₹1,510–₹3,730**, depending on the sensor specifications and supplier.

Industrial-grade pressure, vibration, temperature, and electrical measurement hardware can cost considerably more.

**11. Prototype Implementation**

The prototype is implemented using an ESP8266 NodeMCU, sensors, OLED display, buzzer, breadboard, and jumper wires.

**Step 1 – Controller installation**

The ESP8266 NodeMCU is placed on the breadboard and connected to a regulated power source.

**Step 2 – Temperature monitoring**

The DS18B20 is connected to the ESP8266 and positioned to represent engine-temperature measurement.

**Step 3 – Vibration monitoring**

The vibration sensor is mounted on a prototype generator model or suitable vibration source.

**Step 4 – Oil-pressure monitoring**

A pressure sensor or simulated pressure signal is connected through an appropriate signal-conditioning circuit.

**Step 5 – Fuel monitoring**

A fuel-level sensor or potentiometer can be used to simulate changes in fuel level during prototype testing.

**Step 6 – Battery monitoring**

A safe low-voltage test source and properly designed voltage-divider/ADC interface are used to demonstrate battery monitoring.

**Step 7 – OLED installation**

The OLED is connected through the $I^2C$ interface and displays the measured parameters.

**Step 8 – Alarm installation**

The buzzer is connected to a digital output and activates when the system identifies a warning or critical condition.

**Step 9 – Software implementation**

The program is uploaded using Arduino IDE.

**Step 10 – Wi-Fi/IoT implementation**

The ESP8266 connects to Wi-Fi and sends generator data to a cloud or MQTT platform.

**12. Results & Performance Analysis**

The prototype is expected to provide continuous monitoring of major diesel-generator parameters and generate appropriate alerts when abnormal conditions are detected.

**Test and expected result table**

| Parameter | Normal Condition | Abnormal Condition | Expected Response |
|---|---|---|---|
| Temperature | Below warning level | Excessive temperature | Warning/Critical |
| Vibration | Normal | Excessive vibration | Warning |
| Oil Pressure | Adequate | Low pressure | Critical alarm |
| Fuel Level | Above minimum | Low fuel | Warning |
| Battery Voltage | Normal | Low voltage | Warning |
| Generator Status | Running | Stopped/abnormal | Status update |
| Wi-Fi | Connected | Disconnected | Reconnection attempt |
| Cloud/MQTT | Connected | Connection failure | Local monitoring continues |

**12.1 Temperature Performance**

The system continuously monitors the engine temperature. A temperature increase beyond the configured threshold results in a warning. This provides an early indication of possible overheating.

**12.2 Vibration Performance**

The vibration sensor detects abnormal mechanical movement. Repeated vibration warnings can indicate the need for mechanical inspection.

**12.3 Oil Pressure Performance**

Oil pressure is one of the most important generator parameters. A low-pressure condition is treated as critical because insufficient lubrication can cause significant engine damage.

**12.4 Fuel Level Performance**

Continuous fuel-level monitoring allows operators to identify low fuel conditions before the generator unexpectedly stops.

### 12.5 Battery Performance

Monitoring battery voltage provides an indication of the starting system's condition. A low voltage reading can be used to trigger maintenance action.

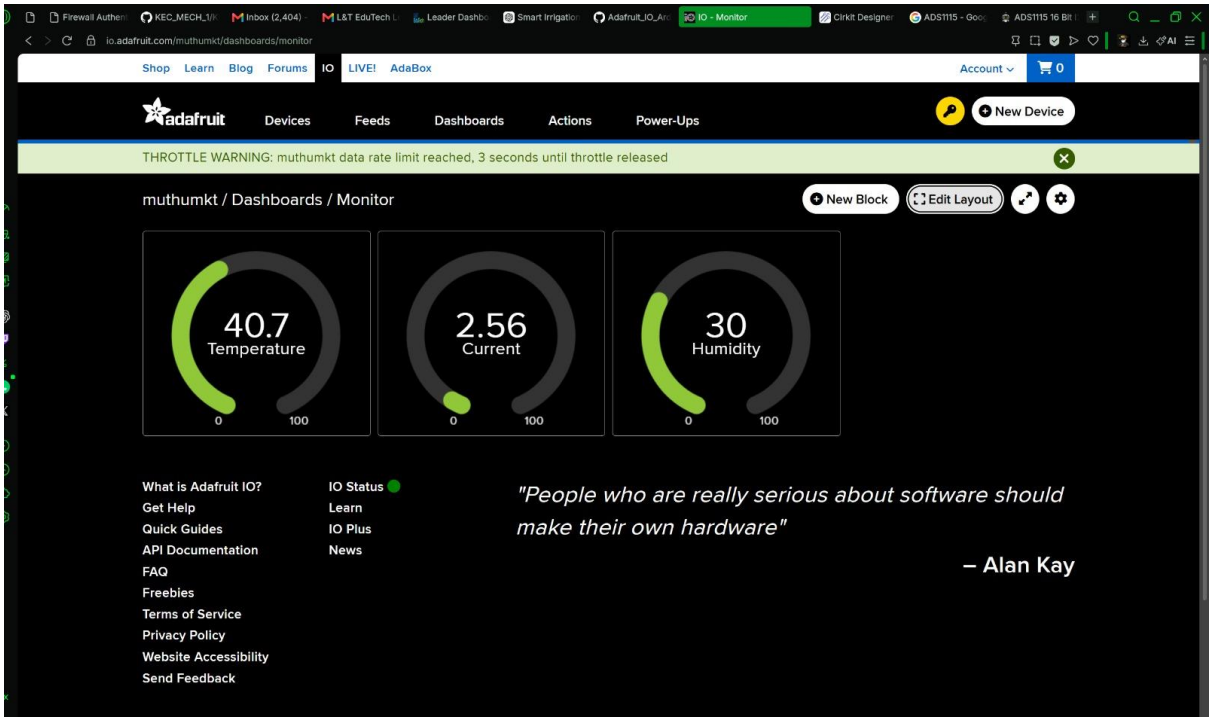
### Conclusion

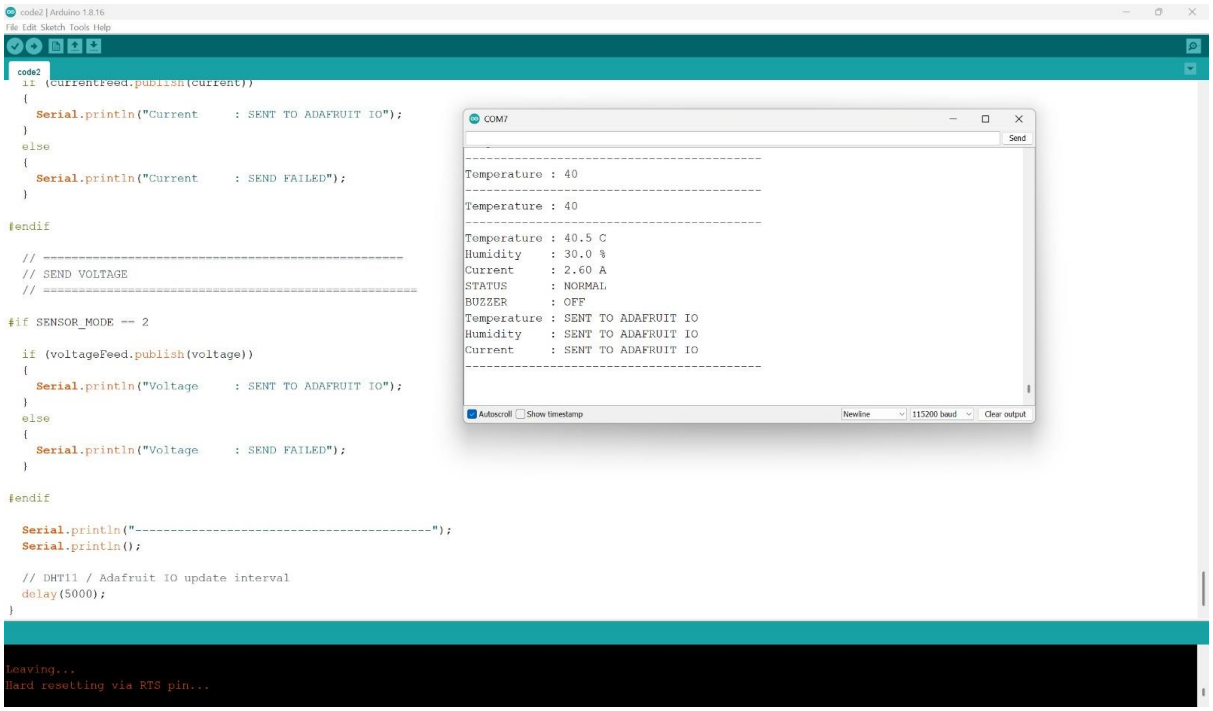
The **Industrial Diesel Generator Monitoring and Maintenance System** provides a practical IoT-based approach for continuously monitoring important generator parameters. The ESP8266 acts as the central controller and receives information from temperature, vibration, oil-pressure, fuel-level, battery-voltage, and generator-status sensors.

The collected information is analyzed against predefined operating limits. The OLED provides local monitoring, while the buzzer provides immediate warning when an abnormal condition occurs. Through Wi-Fi and an optional cloud/MQTT platform, the data can also be monitored remotely.

The system can reduce dependence on manual inspection, improve fault detection, support preventive maintenance, and reduce the possibility of unexpected generator downtime.

OUTPUT:




code2 | Arduino 1.8.16
File Edit Sketch Tools Help
code2
if (currentFeed.publish(current))
{
Serial.println("Current : SENT TO ADAFRUIT IO");
}
else
{
Serial.println("Current : SEND FAILED");
}
#endif
// ======================================================
// SEND VOLTAGE
// ======================================================
#if SENSOR_MODE == 2
if (voltageFeed.publish(voltage))
{
Serial.println("Voltage : SENT TO ADAFRUIT IO");
}
else
{
Serial.println("Voltage : SEND FAILED");
}
#endif
Serial.println("------------------------------------------");
Serial.println();
// DHT11 / Adafruit IO update interval
delay(5000);
}
Leaving...
Hard resetting via RTS pin...
COM7
Send
Temperature : 40
Temperature : 40
Temperature : 40.5 C
Humidity : 30.0 %
Current : 2.60 A
STATUS : NORMAL
BUZZER : OFF
Temperature : SENT TO ADAFRUIT IO
Humidity : SENT TO ADAFRUIT IO
Current : SENT TO ADAFRUIT IO
Autoscroll
Show timestamp
Newline
115200 baud
Clear output